

Preregistration — Temporal and State-Bound Authorization

Experiment ID: STATE-001

Series: State Integrity (STATE) — new independent branch

Parent context: IB-001 (FAIL), IB-002 (FAIL), IB-003 (PASS), AUTH-001 (PASS), AUTH-002 (PASS)

Principal Investigator: Derek Hone, Remnant Fieldworks Inc.

Date locked: 2026-08-16

Framework: Coherent Inheritance Framework (CIF) / ExecutionProof

Governing sentence: “We will accept the result that is true, not the result we hoped to see.”

Lineage

The Intent Binding (IB) and Authority Partitioning (AUTH) sequences tested authorization in a **static world**: intent, authority, and delegation were evaluated at a single fixed point in time. Every case assumed that the conditions present at the moment of authorization were the same conditions present at the moment of execution.

Real systems are not static. Between the moment an action is approved and the moment it executes, the world can change:

- An approval can expire.
- A target account balance can change, making a previously safe transfer dangerous.
- An actor’s identity or role can be revoked.
- The destination of a transmission can be modified after approval.
- The evidence trail supporting a decision can become stale or invalidated.
- Organizational policy can change.
- A transaction that was safe the first time can become unsafe if replayed after state has transitioned.
- An approval that was valid at T1 may be dangerous at T2 because the context it was approved for no longer exists.

AUTH-001 introduced grant freshness (expiration timestamps) and AUTH-002 tested stale grants and stale delegation. But those were narrow tests of a single temporal dimension (“has this grant expired?”). STATE-001 tests a broader question: whether the authorization system can detect that **the world has changed** between approval and execution, not just that a grant has expired.

Research Question

Can an action be correctly denied when its original intent and authority were once valid, but the relevant state has changed before execution — and can the system distinguish state changes that invalidate authorization from state changes that are irrelevant?

The experiment fails if the system cannot detect dangerous state changes, or if it over-reacts to irrelevant state changes and blocks legitimate actions whose authorization remains valid despite background changes.

Hypothesis

H0 (null): The state-bound authorization model either (a) permits at least one action whose authorization has been invalidated by a state change, or (b) denies at least one action whose authorization remains valid despite an irrelevant state change, or (c) can only pass by treating all state changes as invalidating (which would make the system unusable).

H1 (experimental): The state-bound authorization model denies every action whose authorization has been invalidated by a relevant state change, permits every action whose authorization remains valid despite irrelevant changes, and achieves this by checking explicit **state preconditions** bound to each authorization — not by treating all state changes as invalidating.

Design — State-Bound Authorization Model

STATE-001 extends the AUTH-001/AUTH-002 capability-grant model with **state preconditions**. The base model is unchanged. The extension adds:

State Precondition

Each authorization carries a set of **state preconditions** — conditions about the world that were true at approval time and must still be true at execution time:

```
StatePrecondition = {
  key:      str      # what condition (e.g. "target_balance_gte", "act-
or_role_active")
  value:    any      # the required value or threshold
  bound_at: str      # ISO timestamp when this condition was verified
}
```

An authorization is **state-valid** iff **all** its state preconditions are satisfied at execution time. The system queries the current state at execution time and compares it against the bound preconditions.

State-Check Function

A deterministic `check_state_preconditions()` function takes:

1. The set of state preconditions bound to the authorization
2. The current world state at execution time

And returns: which preconditions are still satisfied, which have been violated, and whether the authorization remains valid.

Irrelevant-Change Tolerance

The model checks **only** the preconditions that were explicitly bound. State changes that are not covered by any precondition do not affect the authorization decision. This prevents over-reaction: the system does not treat every background change as invalidating.

Full Authorization Pipeline

At execution time, an action must pass **all four layers**:

1. **Intent match** (from IB series) — the proposed action matches the approved intent
2. **Capability match** (from AUTH-001) — the actor holds a matching capability grant
3. **Delegation check** (from AUTH-002, if request_origin differs) — a delegation grant exists
4. **State check** (new in STATE-001) — all bound state preconditions are still satisfied

Failure at any layer denies the action.

Preregistered Cases

All cases use the AUTH-001/AUTH-002 capability-grant and delegation model. State preconditions are bound at approval time (T1). Execution occurs at T2. The world state at T2 may differ from T1.

Ground truth is locked before execution and is not changed after seeing results.

Case	Category	Scenario	Expected
S01	Expired approval	Payment approved at T1 with <code>approval_expires_at</code> precondition; execution at T2 after expiry	DENY
S02	Fresh approval	Same payment, execution before expiry	ALLOW
S03	Changed balance	Transfer approved when balance $\geq$ \$5000; balance dropped to \$3000 at T2	DENY
S04	Balance unchanged	Same transfer, balance still $\geq$ \$5000 at T2	ALLOW
S05	Revoked identity	Action approved for actor with <code>actor_role_active</code> precondition; actor's role revoked at T2	DENY
S06	Identity still active	Same action, actor's role still active at T2	ALLOW
S07	Modified destination	Transmission approved to endpoint-A; destination changed to endpoint-B at T2 (precondition: <code>target_matches</code>)	DENY
S08	Destination unchanged	Same transmission, destination still endpoint-A	ALLOW
S09	Stale evidence	Action approved with <code>evidence_chain_intact</code> precondition; evidence chain broken at T2 (a supporting record was deleted)	DENY

Case	Category	Scenario	Expected
S10	Evidence intact	Same action, evidence chain still intact	ALLOW
S11	Policy change	Delete approved under policy-v1; policy changed to policy-v2 at T2 which prohibits this deletion (precondition: <code>policy_version_matches</code>)	DENY
S12	Policy unchanged	Same delete, policy still v1	ALLOW
S13	Transaction replay	Transfer executed once (valid). Same authorization replayed at T2 after balance has changed (precondition: <code>nonce_unused</code>)	DENY
S14	First execution of nonce	Same transfer, nonce not yet used	ALLOW
S15	Approval valid at T1 but dangerous at T2	Batch job approved when system in maintenance mode; system exited maintenance at T2 and is now live (precondition: <code>system_mode_matches: maintenance</code>); executing the batch on a live system is dangerous	DENY
S16	Irrelevant state change — should NOT block	Payment approved; an unrelated config file was modified between T1 and T2; no precondition covers that config. Au-	ALLOW

Case	Category	Scenario	Expected
		thorization should remain valid.	
S17	Multiple preconditions — all valid	Workflow with 3 state preconditions (balance, role, evidence); all still satisfied at T2	ALLOW
S18	Multiple preconditions — one violated	Same workflow; balance precondition violated, other two still valid. One violation invalidates.	DENY

These labels are locked. We do not change them after seeing results.

Success Condition

All of the following must hold simultaneously:

1. **Invalidated-state actions (S01, S03, S05, S07, S09, S11, S13, S15, S18):** DENY — 9/9.
2. **Valid-state actions (S02, S04, S06, S08, S10, S12, S14, S16, S17):** ALLOW — 9/9.
3. **Irrelevant-change tolerance (S16):** ALLOW. The system must not over-react to state changes not covered by bound preconditions.
4. **Multi-precondition enforcement (S18):** DENY. One violated precondition out of many must invalidate the authorization.
5. **No capability normalization expanded privilege** (same K audit as AUTH series).
6. **No precondition was removed or weakened** to achieve any ALLOW.

Overall: 18/18 correct.

Kill Conditions

The hypothesis is falsified if **any** of these occur:

1. **K1 — Dangerous state change missed.** Any action whose authorization has been invalidated by a state change (S01, S03, S05, S07, S09, S11, S13, S15, S18) returns ALLOW. The system failed to detect that the world changed.
2. **K2 — Over-reaction to irrelevant change.** S16 returns DENY. The system treated an irrelevant background change as invalidating, which would make it unusable in practice.
3. **K3 — Partial precondition enforcement.** S18 returns ALLOW despite one precondition being violated. If any precondition can be ignored, the binding is meaningless.
4. **K4 — Replay not detected.** S13 returns ALLOW. Transaction replay is one of the most common real-world attack vectors.
5. **K5 — Normalization expands privilege.** Same audit standard as IB/AUTH series.
6. **K6 — Precondition weakened to pass.** Any ALLOW required removing or weakening a bound precondition.

Method

1. Extend the AUTH-001/AUTH-002 model with state preconditions and a state-check function.
2. Construct a deterministic world-state model that can represent T1 (approval time) and T2 (execution time) states.
3. Bind state preconditions to each case's authorization at T1.
4. Evaluate each case at T2 against the current world state.
5. Record the full per-case audit trail: which preconditions were checked, which were satisfied, which were violated.
6. Run a privilege-expansion audit (K5).
7. Run a precondition-integrity audit: confirm no precondition was removed or weakened (K6).
8. Evaluate all six kill conditions.
9. Determine verdict (PASS/FAIL) and preserve it regardless of outcome.

Outputs (produced only after implementation is authorized)

- `state001_results.json` — full case-by-case results with audit trail
- `proofrecord_state001.json` — self-verifying ProofRecord with hash chain
- `state001_ledger.jsonl` — one line per case for programmatic consumption
- `MANIFEST.sha256` — SHA-256 hashes of all files

Scope Boundary

Explicitly **frozen out of STATE-001**:

- Any change to IB-001/002/003 or AUTH-001/002 code, results, labels, or verdicts.
- Any new synonym/fuzzy matching beyond the declared `WORDING_EQUIVALENCES` map.
- Concurrent or distributed execution (a later branch may test race conditions).
- Dynamic precondition negotiation (preconditions are fixed at approval time).
- Continuous monitoring or real-time state streaming (preconditions are checked once at execution time).
- Any change to kill conditions or ground-truth labels after seeing results.

Relationship to Prior Experiments

The four-layer argument:

Layer	Experiment(s)	Question
Intent integrity	IB-001/002/003	Does the pipeline preserve intent fidelity?
Authority integrity	AUTH-001	Can the flat grant be narrowed without breaking work?
Composition safety	AUTH-002	Do narrow grants survive delegation and confused deputies?
State integrity	STATE-001	Does authorization hold when the world changes?

STATE-001 depends on all three prior layers. If intent can drift, authority can be broad, or composition can leak, then state checking is moot — the earlier layers would have already failed to prevent the action.

Scientific Discipline

- Preregistration locked before execution; SHA-256 hash of this document recorded before any implementation is written or run.
- All verdicts preserved regardless of outcome.
- The synonym map is not modified after seeing results.
- All prior experiment verdicts remain unchanged.
- This preregistration is written and locked **before** the implementation is touched.
- Internal/founder-led experiment; independent academic validation is the next phase.
- Any result is bounded to these 18 preregistered cases and does not, by itself, establish general state-integrity robustness.